

CS336 Assignment 2 实验报告

linyun

2026 年 7 月 5 日

1 Benchmarking Script

1.1 实验设置

本实验使用 NVIDIA GeForce RTX 4090，对 small 模型的一次完整训练步骤进行性能测试。实验配置为 batch size 4、context length 512，首先执行 5 次预热，随后进行 10 次正式测量。

1.2 带 warm-up 实验结果

实验在一张 NVIDIA GeForce RTX 4090 (24GB 显存) 上进行。各模型的测量结果如表 1 所示，其中 “-” 表示由于显存不足而未能获得结果。

表 1: 不同规模模型的性能测试结果

模型	Forward (ms)	Backward (ms)	Optimizer (ms)	运行状态
Small	24.451 ± 0.238	54.034 ± 0.244	12.308 ± 0.114	Full step 成功
Medium	74.276 ± 0.264	154.364 ± 0.432	39.441 ± 0.229	Full step 成功
Large	168.974 ± 0.162	339.278 ± 0.465	-	Full step OOM
XL	-	-	-	OOM
10B	-	-	-	OOM

但是，Large 模型在运行包含 AdamW 更新的完整训练步骤时发生显存不足，因此未能获得优化器更新时间。

xl 和 10B 模型在当前硬件配置下无法完成前向传播，直接发生显存不足。

该结果表明，随着模型规模增长，参数、梯度、激活值及优化器状态的显存开销会迅速超过单卡容量，后续需要通过混合精度、激活检查点或参数分片等方法降低显存占用。

对于成功完成的测量，各阶段标准差均明显小于平均值，说明经过 5 次预热后，运行时间较为稳定。反向传播始终是耗时最高的阶段，其耗时约为前向传播的两倍。优化器耗时是三个阶段最少的。

一次完整训练步骤的各阶段耗时之和约为：

$$24.451 + 54.034 + 12.308 = 90.793 \text{ ms.}$$

反向传播耗时约为前向传播的 2.21 倍，是完整训练步骤中耗时最高的阶段。三项测量的标准差均远小于平均值，相对波动均低于 1%，说明经过 5 次预热后运行时间较为稳定。

1.3 改变 Warm-up 次数的实验结果

1.3.1 Warm-up = 0

在不进行预热的情况下，各模型的测量结果如表 2 所示。实验配置保持为 batch size 4、context length 512，并正式测量 10 次。

表 2: 不进行预热时的性能测试结果

模型	Forward (ms)	Backward (ms)	Optimizer (ms)
Small	53.807 ± 87.837	67.246 ± 38.523	17.321 ± 14.372
Medium	125.609 ± 153.896	167.544 ± 40.169	43.418 ± 11.581
Large	197.071 ± 84.114	350.336 ± 33.491	—

与 5 次预热的结果相比，未预热时各阶段的平均耗时普遍更高，标准差也显著增大。其中 Small 和 Medium 模型的 Forward 标准差甚至超过了平均值，说明 10 次测量中存在耗时特别高的初始迭代。

这是因为第一次运行可能包含 CUDA 上下文和动态库初始化、GPU 显存首次分配、内核加载以及 AdamW 状态创建等一次性开销。随着模型重复运行，这些初始化工作不再出现，因此后续迭代明显更快，最终造成较大的测量波动。

1.3.2 Warm-up = 1

执行 1 次预热后的完整训练步骤结果如表 3 所示。实验配置保持为 batch size 4、context length 512，并正式测量 10 次。

Small 模型完整训练步骤的各阶段耗时之和为：

$$24.930 + 55.247 + 12.596 = 92.773 \text{ ms.}$$

Medium 模型完整训练步骤的各阶段耗时之和为：

表 3: 执行 1 次预热时的完整训练步骤性能

模型	Forward (ms)	Backward (ms)	Optimizer (ms)
Small	24.930 ± 1.568	55.247 ± 1.691	12.596 ± 0.268
Medium	74.251 ± 0.254	154.445 ± 0.712	39.613 ± 0.383

$$74.251 + 154.445 + 39.613 = 268.309 \text{ ms.}$$

另外，在 Medium 模型上使用仅包含前向和反向传播的模式进行了一次对照实验，结果如表 4 所示。

表 4: Medium 模型在不同运行模式下的耗时比较

运行模式	Forward (ms)	Backward (ms)
Full step	74.251 ± 0.254	154.445 ± 0.712
Forward + backward	74.617 ± 0.300	154.633 ± 0.517

不进行预热时，测量结果的平均值和标准差都明显增大，例如 Small 模型的 Forward 由 5 次预热后的 24.451 ± 0.238 ms 变为 53.807 ± 87.837 ms，说明初始迭代包含 CUDA 初始化、显存分配、内核加载和 AdamW 状态创建等一次性开销。行 1 次预热后，大部分一次性开销已经被排除，结果接近 5 次预热，但 Small 模型的波动仍然更大。1~2 次预热可能仍不足以让 GPU 频率、缓存、内存分配器和各类延迟初始化完全稳定，因此使用 5 次预热能得到更加可靠的稳态性能结果。

1.4 nsys 性能分析

一个示例的 nsys 命令如下，我真正运行的命令放在 `run_all_nsys.sh` 里，方便批量运行。

```
nsys profile \
  --trace=cuda,nvtx,cudnn,cublas,osrt \
  --pytorch=functions-trace,autograd-shapes-nvtx \
  --sample=none \
  --cpuctxsw=none \
  --output=report_nvtx \
  --force-overwrite=true \
  -- python benchmark_nvtx.py \
  --model-size small \
  --context-length 512 \
```

```
—mode full_step \  
—warmup—steps 5 \  
—measurement—steps 10
```

—sample=none 是关闭 CPU IP/backtrace sampling; —cpuctxsw=none 是关闭线程上下文切换采集;

1.4.1 实验结果分析

(a) What is the total time spent on your forward pass? Does it match what we had measured before with the Python standard library? 对于 small 模型, 使用 python 标准库测量的三种 context length 前向传播时间分别为: 60.130 ± 2.800 ms、 60.141 ± 1.905 ms、 74.061 ± 0.309 ms 差不太多。原因是?

使用 nsys 的平均时间是: 56.996ms+-3.566ms、56.139ms+-1.673ms、55.684ms+-1.435ms; 会比用 python 那边统计的少那么 5ms 左右, 我猜测是 nsys 是直接统计的 kernal 时间所以没有一些通信或者框架的开销?

(b) What CUDA kernel takes the most cumulative GPU time during the forward pass? How many times is this kernel invoked during a single forward pass of your model? Is it the same kernel that takes the most runtime when you do both forward and backward passes? (Hint: look at the “CUDA GPU Kernel Summary” under “Stats System View”, and filter using NVTX ranges to identify which parts of the model are responsible for which kernels.)

对 nvtx.my_forward 的 time line 应用 filter, 然后看看它的 kernal summary, 发现 是 `ampere_sgemm_128x64_tn`, 总用时 7.238 ms 调用次数 85, 时间占比 69.8%——为啥这里一个 forward 用时 55ms, 而 kernal 的总用时只有 10ms 左右呢? ——猜测大概是 nsys 的 profiler 开销 (尤其是追踪 pytorch 函数的), 加上 cpu 调用 api 和 dispatch kernel 的开销, 导致总时间比 kernel 的时间多了很多。因为 nsys 只是统计 kernal 在 gpu 上跑的时间。